

Dado que Dart ejecuta el código en un solo hilo principal (*Single-threaded Event Loop*), realizar operaciones pesadas o de espera (peticiones a APIs REST, consultas a bases de datos SQLite/Isar o lectura de archivos) de forma síncrona bloquearía por completo la interfaz de usuario. La programación asíncrona nos permite delegar estas tareas en segundo plano sin congelar la app.

12.1 El Modelo de Ejecución: *Event Loop* e *Isolates*

Dart utiliza un **Event Loop** (bucle de eventos) que gestiona dos colas de ejecución internas:

1. **Microtask Queue:** Tareas internas prioritarias y de muy corta duración.
2. **Event Queue:** Tareas asíncronas externas (eventos de interfaz, respuesta de temporizadores, I/O de red o disco).

Concepto Clave (*Isolates*): Todo el código de Dart corre por defecto dentro de un *Isolate* principal que posee su propio espacio de memoria aislado. La asincronía **no es multihilo**, sino la capacidad de posponer la respuesta de una tarea mientras el *Event Loop* sigue procesando otros eventos de la interfaz.

12.2 Programación basada en *Future* y *async/await*

Un *Future<T>* representa una promesa de un valor de tipo *T* que estará disponible en algún momento del tiempo (o que puede fallar con un error).

- **async:** Convierte una función tradicional en una función asíncrona que retorna automáticamente un *Future*.
- **await:** Pausa la ejecución de la función asíncrona en esa línea específica hasta que el *Future* se resuelva, **sin bloquear el hilo de la interfaz de usuario**.

```
// Función asíncrona que simula un retraso de red
Future<String> obtenerMatricula() async {
  await Future.delayed(const Duration(seconds: 2));
  return '2026630001';
}

void main() async {
  print('Consultando...');
  final matricula = await obtenerMatricula(); // Espera sin bloquear el Event Loop
  print('Matrícula obtenida: $matricula');
}
```

12.3 Flujos Continuos de Datos con *Stream* y *async**

Mientras que un *Future* entrega **un único valor** en el tiempo, un *Stream<T>* es una tubería que emite **múltiples valores de forma continua a lo largo del tiempo**. En Flutter, los *Stream* son la columna vertebral

de la reactividad, los sockets y arquitecturas de estado como BLoC.

- **Productor (`async*` y `yield`):** Para generar un `Stream`, se utiliza el generador asíncrono `async*` y la instrucción `yield` para emitir cada dato individual hacia la tubería.
- **Consumidor (`await for`):** Para escuchar los valores conforme van llegando, se puede iterar mediante un ciclo `await for` o suscribirse mediante `.listen()`.

```
// Productor de un Stream de conteo
Stream<int> contadorRegresivo(int inicio) async* {
  for (int i = inicio; i >= 0; i--) {
    await Future.delayed(const Duration(seconds: 1));
    yield i; // Emite el valor actual hacia el Stream
  }
}
```

12.4 Ejemplo

Creen el archivo `bin/d12_asincronia.dart` en VS Code. El siguiente programa muestra cómo consumir un endpoint asíncrono con `Future` y cómo procesar un flujo continuo de telemetría mediante `Stream`.

```
/// Módulo D12: Programación Asíncrona en Dart (Future, async/await y Stream)
/// Demostración de consumo de API asíncrona, manejo de excepciones y flujos de
datos.

void main() async {
  print('=====');
  print('  SISTEMA DE MONITOREO Y APIS (D12 DART) - ESCOM  ');
  print('=====');

  // 1. Manejo de Petición Asíncrona con Future (Puntual)
  print('--- 1. Petición Asíncrona HTTP (Future) ---');
  try {
    print(' [HTTP] Enviando solicitud GET a la API de la ESCOM...');
    final perfil = await consultarPerfilEstudiante('2026630123');
    print(' [HTTP] Respuesta recibida con éxito: $perfil\n');
  } catch (e) {
    print(' [HTTP ERROR] $e\n');
  }

  // 2. Manejo de Flujo Continuo con Stream (Reactivo)
  print('--- 2. Transmisión de Datos en Tiempo Real (Stream) ---');
  print(' [SOCKET] Conectando con sensor de tráfico del servidor...\n');

  final Stream<double> flujoCarga = monitorearCargaServidor(3);

  // Consumo del Stream mediante ciclo await for
  await for (final carga in flujoCarga) {
    print(' [TELEMETRÍA $timeFormat]: Carga de CPU al
```

```

    ${carga.toStringAsFixed(1)}%');
  }

  print('\n=====');
}

/// Helper para obtener formato de hora rápido
String get timeFormat => DateTime.now().toIso8601String().substring(11, 19);

/// Simula una petición asíncrona que retorna un Future
Future<String> consultarPerfilEstudiante(String boleta) async {
  // Simula latencia de red de 1.5 segundos
  await Future.delayed(const Duration(milliseconds: 1500));

  if (boleta.isEmpty) {
    throw Exception('La boleta ingresada no es válida.');
```

```

  }

  return 'Estudiante: Ozzy | Carrera: Sistemas Computacionales | Boleta: $boleta';
}

/// Generador de Stream asíncrono (async*) que emite lecturas periódicas
Stream<double> monitorearCargaServidor(int lecturasTotales) async* {
  final valoresSimulados = [34.5, 68.2, 91.0];

  for (int i = 0; i < lecturasTotales; i++) {
    await Future.delayed(const Duration(seconds: 1));
    // Emitimos el valor hacia el Stream mediante yield
    yield valoresSimulados[i % valoresSimulados.length];
  }
}
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d12_asincronia.dart
```

12.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para consolidar la programación asíncrona con `Future` y `Stream`.

Ejercicio 1: Simulación de Login con Manejo de Excepciones Asíncronas

Crea un sistema de autenticación asíncrono:

1. Escribe la función `Future<String> autenticarUsuario(String usuario, String password):`

- Simula un retardo de red de 2 segundos utilizando `Future.delayed`.
 - Si `usuario == 'ozzy'` y `password == 'escom2026'`, retorna un token de sesión ficticio: `'TOKEN_JWT_SECURE_98765'`.
 - Si las credenciales fallan, lanza un `Exception('Credenciales inválidas')`.
2. En la función `main()`, ejecuta dos pruebas con `await` dentro de un bloque `try-catch`: una con credenciales correctas y otra con incorrectas, capturando y mostrando los errores.

Ejercicio 2: Descarga de Archivos Porcentual con Stream

Escribe una simulación de descarga de paquetes de datos:

1. Crea la función `Stream<int> descargarPaquete(int tamanoTotalMB)` usando `async*`:
 - Cada 500 milisegundos, simula la descarga de 20 MB.
 - Emite mediante `yield` el porcentaje descargado $((\text{descargado} / \text{total}) \times 100)$.
 - Detén la emisión cuando se alcance el 100%.
 2. En `main()`, consume el `Stream` para un archivo de 100 MB e imprime en consola el avance porcentual segundo a segundo.
-